

EXPERIMENT 6 – Semaphore Implementation

Aim: To implement Mutual Exclusion using Semaphore.

C Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <semaphore.h>
#include <sys/mman.h>
#include <sys/wait.h>

int main()
{
    sem_t *sem;

    sem = mmap(NULL, sizeof(sem_t),
               PROT_READ | PROT_WRITE,
               MAP_SHARED | MAP_ANONYMOUS, -1, 0);

    sem_init(sem, 1, 1);

    if(fork() == 0)
    {
        sem_wait(sem);
        printf("Child Process Entering Critical Section\n");
        sleep(3);
        printf("Child Process Leaving Critical Section\n");
        sem_post(sem);
        exit(0);
    }

    sem_wait(sem);
    printf("Parent Process Entering Critical Section\n");
    sleep(3);
    printf("Parent Process Leaving Critical Section\n");
    sem_post(sem);

    wait(NULL);
    sem_destroy(sem);

    return 0;
}
```

Compilation: `gcc semaphore.c -o semaphore`

Output:

```
Parent Process Entering Critical Section
Parent Process Leaving Critical Section
Child Process Entering Critical Section
Child Process Leaving Critical Section
```

Shell Script (Lock file simulation):

```
#!/bin/bash

LOCKFILE="/tmp/mylock"

while [ -f "$LOCKFILE" ]
do
    sleep 1
done

touch "$LOCKFILE"

echo "Entering Critical Section"
sleep 5
echo "Leaving Critical Section"

rm -f "$LOCKFILE"
```

Result: Mutual Exclusion using Semaphore was implemented successfully.
